

Résumé des commandes shell sous Linux

I. Rappel : Fonctionnement du « shell »

Une fois que l'utilisateur est connecté sur une console, le shell affiche l'invite de commandes (*prompt*). Il se comporte ensuite comme une boucle infinie, à chaque saisie d'instruction :

- affichage de l'invite de commande ;
- lecture de la commande ;
- analyse de la syntaxe ;
- substitution des caractères spéciaux ;
- exécution de la commande ;
- affichage de l'invite de commande ;
- etc.

La séquence de touche kbd:[CTRL+C] permet d'interrompre une commande en cours d'exécution.

L'utilisation d'une commande respecte généralement cette séquence :

Séquence d'une commande

commande [option(s)] [arguments(s)]

- Le nom de la commande est **souvent en minuscules**.
- Un espace sépare chaque élément.
- Les **options courtes** commencent par un tiret (-), alors que les **options longues** commencent par deux tirets (--list)

II. Les commandes générales

1. **La commande apropos** : permet de lister les manuels dont la description comprend les mots passés en arguments.
2. **La commande whatis** affiche la description de la commande passée en argument :

```
$ whatis clear
clear (1) - clear the terminal screen
```

3. **La commande man** permet la lecture du manuel
4. **La commande uname** - Afficher des informations sur le système
5. **La commande shutdown** permet de stopper électriquement, immédiatement ou après un certain laps de temps, un serveur Linux.

Syntaxe de la commande shutdown
[root]# shutdown [-h] [-r] heure [message]

```
[root]# shutdown -h 0:30 "Arrêt du serveur à 0h30"
```

6. **La commande history** permet d'afficher l'historique des commandes qui ont été saisies par l'utilisateur. Les commandes sont mémorisées dans le fichier `.bash_history` du répertoire de connexion de l'utilisateur.
7. **La commande uname** permet de récupérer des informations sur le système comme le nom d'hôte, le processeur de notre machine, la version du noyau... L'option `"-a"` permet de regrouper l'ensemble des options disponibles sur la commande **"uname"**.
8. **L'auto-complétion** permet de compléter les commandes, les chemins saisis ou les noms de fichiers. Un appui sur la touche `kbd:[TAB]` complète la saisie dans le cas d'une seule solution. Sinon, il faudra faire un deuxième appui pour obtenir la liste des possibilités.

III. Affichage et identification

1. **La commande clear** : permet d'effacer le contenu de l'écran du terminal. `[CTRL+L]` aura le même effet que la commande `clear`

2. **La commande echo** permet d'afficher une chaîne de caractères.

```
$ echo « Hello »
```

3. **La commande date** permet d'afficher la date et l'heure.

```
$ date
```

4. **La commande id** affiche le nom de l'utilisateur courant et ses groupes ou ceux d'un utilisateur, si le login de celui-ci est fourni comme argument.

```
$ id util1
```

5. **La commande whoami** affiche le login de l'utilisateur courant.
6. **La commande who** seule affiche le nom des utilisateurs connectés

IV. Arborescence de fichiers

- Sous Linux, l'arborescence des fichiers se présente sous la forme d'un arbre inversé, appelé arborescence hiérarchique unique, dont la racine est le répertoire `« / »`.
 - Le répertoire courant est le répertoire où se trouve l'utilisateur.
 - Le répertoire de connexion est le répertoire de travail associé à l'utilisateur. Les répertoires de connexion sont, en standard, stockés dans le répertoire `/home`.
 - À la connexion de l'utilisateur, le répertoire courant est le répertoire de connexion.
- Un chemin absolu référence un fichier depuis la racine en parcourant l'arborescence complète jusqu'au niveau du fichier :
`/home/groupeA/alice/monfichier`

- Le chemin relatif référence ce même fichier en parcourant l'arborescence complète depuis le répertoire courant :
`../alice/monfichier`
 - Dans l'exemple précédent, les « .. » font référence au répertoire parent du répertoire actuel
 - Un répertoire, même s'il est vide, contiendra obligatoirement au minimum deux références :
 - « . » référence sur lui-même.
 - « .. » référence le répertoire parent du répertoire actuel.
 - Un chemin relatif peut ainsi commencer par « ./ » ou par « ../ ». Lorsque le chemin relatif fait référence à un sous dossier ou à un fichier du répertoire courant, alors le « ./ » est souvent omis.
1. **La commande pwd** (Print Working Directory) affiche le chemin absolu du répertoire courant.
`$ pwd`
`/home/stagiaire`
 2. **La commande cd** (Change Directory) permet de changer le répertoire courant, autrement dit, de se déplacer dans l'arborescence.
`$ cd /tmp`
 3. **La commande ls** affiche le contenu d'un répertoire.
`ls [-a] [-i] [-l] [repertoire1] [repertoire2] [...]`
`$ ls /home`
`.` `..` stagiaire

Options principales

-a	Affiche tous les fichiers, même ceux cachés. Les fichiers cachés sous Linux sont ceux qui commencent par un « . ».
-i	Affiche les numéros d'inode (information contenu dans le système de fichiers)
-l	Affiche sous forme de liste verticale la liste des fichiers avec des informations supplémentaires formatées par colonnes.

Des alias sont fréquemment positionnés au sein des distributions courantes. C'est le cas de l'alias `ll` de la commande `ls -l`

```
alias ll='ls -l --color=auto'
```

4. **La commande mkdir** crée un répertoire ou une arborescence de répertoires.
Syntaxe de la commande mkdir

```
mkdir [-p] repertoire [repertoire] [...]
```

Exemple :

```
$ mkdir /home/stagiaire/travail
```

5. **La commande touch** modifie l'horodatage d'un fichier ou crée un fichier vide si le fichier n'existe pas.

Syntaxe de la commande touch

```
touch [-t date] fichier
```

Exemple :

```
$ touch /home/stagiaire/fichier
```

6. **La commande rmdir** supprime un répertoire vide.

Exemple :

```
$ rmdir /home/stagiaire/travail
```

7. **La commande rm** supprime un fichier ou un répertoire.

Syntaxe de la commande rm

```
rm [-f] [-r] fichier [fichier] [...]
```

La suppression d'un dossier à l'aide de la commande rm, que ce dossier soit vide ou non, nécessitera l'ajout de l'option -r.

8. **La commande mv** déplace et renomme un fichier.

Syntaxe de la commande mv

```
mv fichier [fichier ...] destination
```

Exemples :

```
$ mv /home/stagiaire/fic1 /home/stagiaire/fic2
```

9. **La commande cp** copie un fichier.

Syntaxe de la commande cp

```
cp fichier [fichier ...] destination
```

Exemple :

```
$ cp -r /home/stagiaire /tmp
```

V. Visualisation

1. **La commande file** affiche le type d'un fichier.

Syntaxe de la commande file

```
file fichier [fichiers]
```

Exemple :

```
$ file /etc/passwd /etc  
/etc/passwd: ASCII text  
/etc: directory
```

2. **La commande more** affiche le contenu d'un ou de plusieurs fichiers écran par écran.

Syntaxe de la commande more

```
more fichier [fichiers]
```

Exemple :

```
$ more /etc/passwd
root:x:0:0:root
```

3. **La commande less** affiche le contenu d'un ou de plusieurs fichiers. La commande less est interactive et possède des commandes d'utilisation qui lui sont propres.

Syntaxe de la commande less

```
less fichiers [fichiers]
```

4. **La commande cat** concatène (mettre bout à bout) le contenu de plusieurs fichiers et affiche le résultat sur la sortie standard.

Syntaxe de la commande cat

```
cat fichier [fichiers]
```

Exemple Afficher le contenu d'un fichier vers la sortie standard :

```
$ cat /etc/passwd
```

5. **La commande tac** fait quasiment l'inverse de la commande cat. Elle affiche le contenu d'un fichier en commençant par la fin (ce qui est particulièrement intéressant pour la lecture des logs !).

Exemple : Afficher un fichier de logs en affichant en premier la dernière ligne :

```
[root]# tac /var/log/messages | less
```

6. **La commande head** affiche le début d'un fichier.

Syntaxe de la commande head

```
head [-n x] fichier
```

- n x Affiche les x premières lignes du fichier

7. **La commande tail** affiche la fin d'un fichier.

Syntaxe de la commande tail

```
tail [-f] [-n x] fichier
```

8. **La commande wc** compte le nombre de lignes, mots et/ou octets d'un fichier.

Syntaxe de la commande wc

```
wc [-l] [-m] [-w] fichier [fichiers]
```

Options

-c	Compte le nombre d'octets.
-m	Compte le nombre de caractères.
-l	Compte le nombre de lignes.
-w	Compte le nombre de mots.

9. **La commande sort** trie les lignes d'un fichier. Elle permet d'ordonner, ranger dans un ordre donné, le résultat d'une commande ou le contenu d'un fichier, selon un ordre numérique, alphabétique, par ordre de grandeur (Ko, Mo, Go) ou dans l'ordre inverse.

Syntaxe de la commande sort

```
sort [-kx] [-n] [-o fichier] [-ty] fichier
```

Exemple :

```
$ sort -k3 -t: -n /etc/passwd
root:x:0:0:root:/root:/bin/bash
adm:x:3:4:adm:/var/adm:/sbin/nologin
```

Options

-kx	Précise la colonne x sur laquelle se fera le tri
-n	Demande un tri numérique
-o fichier	Enregistre le tri dans le fichier précisé
-ty	Précise le caractère séparateur de champs y

10. La commande cut permet d'afficher des zones spécifiques d'un fichier.**Exemple :**

```
$cut -c1 /etc/passwd
```

affichera la première colonne du fichier /etc/passwd

Option	Signification
-c1-5	Permet de sélectionner les colonnes 1 à 5.
-c14-	Permet de sélectionner de la colonne 14 à la dernière.
-c1-3,14-18	Permet de spécifier plusieurs plages de colonnes.

On peut également spécifier un *séparateur de champs* avec l'option -d.

Exemple :

```
$cut -d: -f6 /etc/passwd
```

affichera le 6^{ème} champ du fichier /etc/passwd, dont le séparateur de champs est le caractère double point (``:``).

VI. Recherche**1. La commande find** recherche l'emplacement de fichier ou de répertoire.*Syntaxe de la commande find*

```
find repertoire [-name nom] [-type type] [-user
```

Les options de la commande `find` étant très nombreuses, il est préférable de se référer au `man`. Si le répertoire de recherche n'est pas précisé, la commande `find` cherchera à partir du répertoire courant.

L'option `-exec` de la commande `find` Il est possible d'utiliser l'option `-exec` de la commande `find` pour exécuter une commande à chaque ligne de résultat :

```
$ find /tmp -name *.txt -exec rm -f {} \;
```

2. La commande `whereis` recherche des fichiers liés à une commande.

Syntaxe de la commande `whereis`

```
whereis [-b] [-m] [-s] commande
```

Exemple :

```
$ whereis -b ls
```

```
ls: /bin/ls
```

Options

<code>-b</code>	Ne recherche que le fichier binaire.
<code>-m</code>	Ne recherche que les pages de manuel.
<code>-s</code>	Ne recherche que les fichiers sources.

3. La commande `grep` recherche une chaîne de caractères dans un fichier.

Syntaxe de la commande `grep`

```
grep [-w] [-i] [-v] "chaîne" fichier
```

Exemple :

```
$ grep -w "root:" /etc/passwd
```

```
root:x:0:0:root:/root
```

Options

<code>-i</code>	Ignore la casse de la chaîne de caractères recherchée.
<code>-m</code>	Exclut les lignes contenant la chaîne de caractère.
<code>-s</code>	Recherche exactement le mot.

VII. Les méta-caractères

1. Les caractères spéciaux

Les méta-caractères se substituent à un ou plusieurs caractères (voire à une absence de caractère) lors d'une recherche. Ils sont combinables. Les méta-caractères permettent des recherches plus complexes en remplaçant tout ou partie d'un mot. Il suffit de remplacer les inconnues par ces caractères spéciaux.

- a. Le caractère `*` remplace une chaîne composée de plusieurs caractères quelconques. Le caractère `*` peut également représenter une absence de caractère.

```
$ find /home -name "test*"
/home/stagiaire/test
/home/stagiaire/test1
/home/stagiaire/test11
/home/stagiaire/tests
/home/stagiaire/test362
```

- b. Le caractère « ? » remplace un unique caractère, quel qu'il soit.

```
$ find /home -name "test?"  
/home/stagiaire/test1  
/home/stagiaire/tests
```

- c. Les crochets « [] » permettent de spécifier les valeurs que peut prendre un unique caractère.

```
$ find /home -name "test[123]*"  
/home/stagiaire/test1  
/home/stagiaire/test11  
/home/stagiaire/test362
```

Prendre soin de toujours encadrer les mots contenant des méta-caractères par des « " » soit précéder juste le caractère spécial d'un backslash \.

- d. [!...] un caractère quelconque hors de l'ensemble.
- e. Le caractère tilde (« ~ ») permet de repérer un nom de connexion:
- « ~ » seul pour le répertoire de connexion de l'utilisateur;
 - ~nom pour le répertoire de connexion de l'utilisateur nom;
 - Par exemple, ls ~domy/public liste le sous-répertoire public de domy.

Attention !

Ne pas confondre les méta-caractères du shell avec ceux utilisés dans les expressions régulières pour rechercher des patterns particuliers dans des fichiers (utilisées avec les commandes grep, sed et awk).

2. Les expressions régulières

Les expressions régulières sont des critères de recherche. On ne cherche pas un mot précis, mais des suites de caractères (un motif) correspondant aux critères demandés. Elles sont construites comme des opérations arithmétiques, en utilisant des opérateurs divers pour combiner des expressions. Elles sont d'un usage fréquent avec grep bien sûr, mais aussi avec d'autres commande comme sed et awk, La liste des symboles utilisables pour grep sont :

.	un seul caractère quelconque
*	répétition, zéro ou plusieurs fois, du caractère précédent. ab* représente l'ensemble { a,ab,abb,abbb,...}
+	répétition (au moins une fois) du caractère précédent ab+ représente l'ensemble { ab,abb,abbb,...}
?	l'élément précédent est facultatif ab?c représente l'ensemble {ac,abc}
^	début de ligne ^a représente toutes les lignes qui commencent par 'a'
\$	fin de ligne a\$ représente toutes les lignes qui finissent par 'a'
[...]	liste ou intervalle de caractères recherchés [abcd] représente l'ensemble {a,b,c,d} [a-d] représente l'ensemble {a,b,c,d}
[^ab]	listes des caractères interdits

VIII. Redirections et tubes

Sur les systèmes UNIX et Linux, les flux standards sont aux nombres de trois. Ils permettent aux programmes, via la bibliothèque `stdio.h` de faire entrer ou sortir des informations. Par défaut :

- le clavier est le périphérique d'entrée pour le canal 0, appelé `stdin` ;
- l'écran est le périphérique de sortie pour les canaux 1 et 2, appelés `stdout` et `stderr`.

1. La redirection d'entrée

Il est possible de rediriger le flux d'entrée depuis un autre fichier avec le caractère inférieur `<` ou `<<`. La commande `<` lira le fichier au lieu du clavier :

Exemple :

C'est quoi la différence avec `cat notes.csv` ?

- Si vous écrivez `cat notes.csv`, la commande `cat` reçoit en entrée le nom du fichier `notes.csv` qu'elle doit ensuite se charger d'ouvrir pour afficher son contenu.
- Si vous écrivez `cat < notes.csv`, la commande `cat` reçoit le contenu de `notes.csv` qu'elle se contente simplement d'afficher dans la console. C'est le shell qui se charge d'envoyer le contenu de `notes.csv` à la commande `cat`.

Le double chevron ouvrant `<<` permet de lire progressivement depuis le clavier.

```
ines@ines-VirtualBox:~$ sort -n <<FIN
> 12
> 13
> 65
> FIN
12
13
65
```

2. La redirection de sortie

Les sorties standards peuvent être redirigées vers d'autres fichiers grâce aux caractères > ou >>.

La redirection simple > écrase le contenu du fichier de sortie :

```
$ date > fic_date
```

alors que la redirection double >> ajoute (concatène) au contenu du fichier de sortie.

```
$ date >> fic_date
```

Dans les deux cas, le fichier est automatiquement créé lorsqu'il n'existe pas.

La sortie d'erreur standard peut être également redirigée vers un autre fichier. Cette fois-ci, il faudra préciser le numéro du canal (qui peut être omis pour les canaux 0 et 1) :

```
$ ls -R / 2> fic_erreurs
```

```
$ ls -R / 2>> fic_erreurs
```

Exemples de redirections

Redirection de 2 sorties vers 2 fichiers :

```
$ ls -R / >> fic_ok 2>> fic_nok
```

3. Les tubes (pipe)

Un tube (pipe en anglais) est un mécanisme permettant de relier la sortie standard d'une première commande vers l'entrée standard d'une seconde.

Cette communication est monodirectionnelle et se fait grâce au symbole |.

Le symbole pipe "|" est obtenu en appuyant simultanément sur les touches kbd:[AltGR + 6].

Exemples :

```
$ ls -lia / | head # N'afficher que le début
```

```
$ ls -lia / | tail # N'afficher que la fin
```

```
$ ls -lia / | sort # Trier le résultat
```

```
$ ls -lia / | wc # Compter le nombre de mots / caractères
```

IX. Droits d'accès aux fichiers

Chaque fichier du système est associé à des droits d'accès. Les droits d'accès de base sont :

- r (read) : droit de lire le fichier

- w (write) : droit d'écrire dans le fichier
- x (execute): droit d'exécuter le fichier (programme ou script)

Ces droits d'accès sont affichés par la commande `ls -l`. Les droits seront affichés dans l'ordre **ugo** (user, group, other). Si l'un des droits n'est pas accordé, un tiret est affiché à sa place.

	droits concernant le <u>propriétaire</u> du fichier			droits concernant les autres membres du même groupe que le <u>groupe propriétaire</u> du fichier			droits concernant tous les <u>autres</u> utilisateurs		
type de fichier	u(ser)			g(roup)			o(ther)		
-	r	w	x	r	w	-	r	-	-
- indique un fichier ordinaire	Le propriétaire peut <u>lire</u> , <u>écrire</u> et <u>exécuter</u> ce fichier			Les membres du même groupe propriétaire peuvent <u>lire</u> et <u>écrire</u> , mais pas exécuter ce fichier			Les autres utilisateurs peuvent seulement <u>lire</u> ce fichier		

le type de
fichier peut
être :

- ‘-’ pour un fichier;
- ‘d’ pour un dossier;
- ‘s’ pour un dossier particulier ("sticky bit" empêchant qu'un fichier soit supprimé sans l'autorisation de tous les fichiers ou sous-dossiers appartenant à quelqu'un d'autre);
- ‘l’ pour un lien vers un fichier;
- ‘b’ ou ‘c’ pour des fichiers représentant des périphériques (voir dans /dev)

1. La commande **chmod**

Pour gérer ces droits, on peut utiliser la commande `chmod`. Il existe deux modes de modification de droits :

- **le mode littéral** : les catégories ugo et a (all, pour tous), les caractères rwx

Exemples :

- **`chmod u+x nomdufichier`** rend le fichier exécutable (x) pour l'utilisateur propriétaire (u) du fichier.
- **`chmod g-w nomdufichier`** rend le fichier non-modifiable (writable) (w) pour le groupe propriétaire (g) du fichier.

- **chmod o+r nomdufichier** rend le fichier lisible (readable) (r) pour les autres (other) (o).
 - En l'absence de précision sur les utilisateurs concernés: **chmod +w nomdufichier** rend le fichier modifiable (writable) pour l'utilisateur propriétaire, le groupe propriétaire et les autres.
- **le mode numérique (en octal)** : C'est de loin le mode le plus rapide pour fixer des droits. En revanche, lorsqu'on souhaite seulement ajouter/retrancher un droit au propriétaire, au groupe_propriétaire ou aux autres sans connaître les droits actuels, le mode "littéral" est plus adapté.

u			g			o		
r	w	x	r	w	-	r	-	-
2 ²	2 ¹	2 ⁰	2 ²	2 ¹	2 ⁰	2 ²	2 ¹	2 ⁰
4	2	1	4	2	0	4	0	0
7			6			4		

Exemple :

Considérons le fichier resolv.conf dont les droits sont rw-r--r-- soit rw- r-- r--. Pour passer à -wx r-x rwx, il faut taper :

chmod 357 resolv.conf

3 = 2+1 pour -wx

5 = 4+1 pour r-x

7 = 4+2+1 pour rwx

Pour changer les droits sur tous les fichiers du dossier courant de la même façon, on tapera: **chmod 357 * # pour tous les fichiers.**

Et pour changer les droits des fichiers du dossier courant et des sous dossiers de la même façon, on tapera: **chmod 357 * -R # -R pour de façon Récursive.**

2. La commande umask

Les droits par défaut d'un nouveau fichier sont définis par rapport à un masque des droits défini pour chaque utilisateur avec la commande `umask`. La commande `umask` permet donc d'afficher ou de modifier le masque de création du fichier de l'utilisateur.

```
$ umask 0022
```

```
$ umask -S
```

```
u=rwx,g=rx,o=rx
```

Un fichier est toujours créé par un programme : une commande (`touch`, `cat`, ...), un éditeur (`vim`, `kwrite`, ...), un compilateur (`gcc`), ou tout autre application (`konqueror`, `syslog`, ...).

Le programme utilisé définit les droits pour le fichier à créer, par exemple :

- `rw-rw-rw-` (666) pour des fichiers réguliers (non exécutable)
- `rw-rwxrwx` (777) pour des fichiers exécutables

Le système applique ensuite le masque défini par `umask` pour créer les droits du fichier :

droits d'origine (666)	r	w	-	r	w	-	r	w	-
masque (022)	0	0	0	0	1	0	0	1	0
opération droits d'origine & ~masque	↓	↓	↓	↓	X	↓	↓	X	↓
droits du fichier nouvellement créé	r	w	-	r	-	-	r	-	-

Soit l'opération suivante : $666 \& \sim 022 = 644 = rw- r-- r--$

3. Copie d'un fichier

C'est le même principe qui est appliqué en utilisant cette fois les droits du fichier source.

Si le fichier destination existe (écrasement), le masque n'est pas utilisé et à la place on utilise les droits du fichier destination : droits fichier source & droits fichier destination. Il existe des options (`p a`, ...) qui modifient ce comportement et permettent de préserver les propriétés du fichier source.

4. Droits d'accès aux répertoires

Dans le cas des répertoires, l'interprétation des droits est légèrement différente de celle concernant les fichiers. Les informations concernant un répertoire sont obtenues par la commande : `ls -dl <Nom repertoire>`

Exemple

```
$ ls dl /tmp
```

```
drwxrwxrwt 8 root root 4096 20090905 16:46 /tmp/
```

L'interprétation des droits d'accès pour les répertoires est la suivante :

- **r**: autorise la lecture du contenu du répertoire (permet donc de voir la liste des fichiers qui sont dans le répertoire)
- **x**: autorise l'accès au répertoire (droit de traverser et de se déplacer par exemple avec la commande `cd`)
- **w**: autorise la création, la suppression et le changement du nom d'un élément du répertoire.

Remarque :

La permission de supprimer un fichier n'est pas attribuée au fichier lui-même, mais elle dépend uniquement du droit d'écriture dans le répertoire qui le contient.

Permission de supprimer, le bit t (sticky bit)

La permission de supprimer un fichier n'est pas attribuée au fichier lui-même, mais elle dépend uniquement du droit d'écriture dans le répertoire qui le contient. Cela pose un problème pour des répertoires « partagés ».

Par exemple pour le répertoire `/tmp`, tout utilisateur doit pouvoir créer des fichiers dans ce répertoire. Les droits pour ce répertoire seront `rwX rwX rwX`.

L'ensemble des utilisateurs (type d'utilisateur `other`) pourront donc lire et écrire dans `/tmp`. Mais un utilisateur pourrait supprimer un fichier créé par un autre utilisateur. Le bit `t` permet de régler ce problème : un utilisateur ne pourra supprimer un fichier que s'il en est propriétaire.

Le bit `t` apparaît à la place du bit `x` de `other` :

```
$ ls -dl /tmp
```

#si le bit `x` est positionné :

drwxrwxrwt 8 root root 4096 20090905 16:46 /tmp/